1. Explain what ORM is
ORM (Object-Relational Mapping) maps Java objects to relational database tables. It allows developers to work with database records through Java objects instead of manually writing SQL for every operation.
Java class ↔ Database table
Java object ↔ Database row
Java field ↔ Database column

2. What is the difference between JPA and Hibernate?
JPA is a standard that defines how Java applications manage database persistence. Hibernate is an implementation of JPA that handles actual database operations, such as generating SQL and mapping query results to Java objects.In short, JPA defines the rules, while Hibernate implements them.

3. What problem does Spring Data JPA solve?

Spring Data JPA reduces repetitive database operations by automatically providing repository implementations. By extending JpaRepository, developers can use methods like save(), findById(), and findAll() without implementing them manually.
Spring Data JPA uses JPA, which is typically implemented by Hibernate, to access the database.

4. What is an Entity?
An Entity is a Java class managed by JPA and mapped to a database table.
@Entity: Marks the class as a JPA entity.
@Table: Specifies the database table name.
@Id: Defines the primary key.
@GeneratedValue: Specifies how the primary key is generated.

5. Why does an Entity need a primary key?
A primary key uniquely identifies each record in a database table. The @Id annotation defines the primary key of an Entity.
A newly created Entity may not have an ID before being saved because the database can generate the ID during insertion. We use Long instead of long because Long can be null, representing an Entity that has not been saved yet.

6. Why do Student and Course need Enrollment?
We use an Enrollment entity because the relationship between Student and Course has its own attributes. A simple @ManyToMany relationship cannot easily store additional information about the relationship.
For example, Enrollment can store enrolledAt, status, and grade. It represents the relationship between a Student and a Course as a separate entity.

7. What is the owning side?
The Enrollment side is the owning side because it contains the foreign key relationship.
The student_id column in the enrollments table is the foreign key referencing the primary key in the students table.
The owning side is responsible for maintaining the foreign key in the database.

8. What does mappedBy = "student" mean?

mappedBy = "student" indicates that the relationship is managed by the student field in the Enrollment entity.
The value "student" refers to the Java field name, not the database column name student_id. It tells JPA that both sides represent the same relationship and prevents creating an additional relationship mapping.

9. What is the difference between LAZY and EAGER?
LAZY loads related entities only when they are accessed, while EAGER loads related entities immediately when the main entity is loaded.
Relationships should generally use LAZY to avoid loading unnecessary data and improve performance.

10. How does a request reach the database? HTTP Request → Controller → Service → Repository → Hibernate → JDBC → Database
1.HTTP Request: The client sends a request, such as GET /students/1/enrollments, to retrieve the enrollments of the student with ID 1.
2.Controller: Receives the request and extracts the studentId (1).
3.Service: Handles the business logic and calls the Repository to retrieve the student's enrollments.
4.Repository: Provides database access methods, such as findByStudentId(1L).
5.Hibernate: Generates the SQL query based on the Entity mappings.After the database returns the results, Hibernate maps them into Java Enrollment objects.
6.JDBC: Executes the SQL query against the database.
7.Database: Executes the query and returns the matching enrollment records.
In short, the Controller receives the request, the Service handles the business logic, the Repository accesses the data, Hibernate generates SQL and maps the results into Java objects, and JDBC executes the SQL against the database.